

LAB01: DATA & MEMORY

CONTENT

CORE Guidelines & Naming conventions	2
naming conventions	2
Core guidelines	2
Exercises.....	3
Ex00: prepare your folder structure	3
create a folder for this lab	3
Creating your projects	3
Ex01: HANDS-ON DEMO: Variables.....	3
project setup	3
Hands-on demo	3
Exercise: finish the demo code.....	4
Ex02: Expressions	5
project setup	5
Binary operators +, -.....	5
Binary operators *, /	5
Compound expressions	5
Compound assignment and Unary operators	6
Ex03: Making Friends.....	6
part 2: favorite color	6
Ex04: Applications	7
Ex05: Awaits human review.....	8
OPTIONAL: MAKING FRIENDS CONTINUED – ANSI CODES.....	8
Ex06: SDLAndOpenGL (second lab block, hands-on demo).....	9
Ex07: Coding Craftsmanship Challenge	10
Instructions.....	10
Wall of Fame	10
Submission instructions	10
Before zipping	10
Submit	10
References	11
Structure of a program	11
cout, cin, endl, std, using	11
Literals	11
Operator precedence and associativity	11

OBJECTIVE

At the end of these exercises, you should be able to:

- Create a Visual Studio application
- Define and use breakpoints in Visual Studio
- Explain what the Visual Studio build does and how to solve the reported errors
- Explain what an include directive does and know which ones are necessary to use `std::cin`, `std::cout`.
- Use `std::cin` and `std::cout` for console input and output
- Define comment lines in code
- Define and initialize different types of variables: `int`, `double`, `char`, `bool`
- Create literals of each type
- Assign another value to a variable
- Use operators with these types
- Add given code file to a project
- Find and solve errors and warnings related to the above topics
- Explain the basic game loop
- Draw basic graphic shapes on the window. (lines, rectangles, ellipses)

We advise you to **make your own summary of topics** which are new to you.

CORE GUIDELINES & NAMING CONVENTIONS

NAMING CONVENTIONS

The **naming conventions** used in Prog1 and Prog2 promote a consistent coding style and should be followed, just as you will need to follow the conventions of any project, team, or company you end up with.

Category	Naming Convention / Pattern	Code Example
Local Variables	lowerCamelCasing	<code>float width{};</code> <code>const float innerSize{ 10.f };</code>
Boolean Identifiers	Verb indicating boolean state (Is..., Has...)	<code>bool isReady{};</code> <code>bool IsKeyPressed();</code>

CORE GUIDELINES

Always check the [PROG1: Naming Conventions & C++ Core Guidelines](#) document for a detailed explanation about the guidelines below.

- ✓ **ES.10;** Declare one name (only) per declaration
- ✓ **ES.20;** Always initialize an object
- ✓ **ES.23;** Prefer the `{}` initializer syntax
- ✓ **NL.5;** Don't encode type information in names
- ✓ **NL.8;** Use a consistent naming style
- ✓ **NL.20;** Don't place two statements on the same line
- ✓ **NR.1;** Don't force all declarations to the top

EXERCISES

EX00: PREPARE YOUR FOLDER STRUCTURE

By keeping all your exercises structured it will be much easier to find them again, for example when we refer to them in later exercises or you want to look something up.

CREATE A FOLDER FOR THIS LAB

Each lab you should **create a folder** with the correct name:

- **L01_Name_FirstName**
 - Replace **Name** and **FirstName** by your personal last and first name
 - Avoid any special characters like ` or - by replacing them with an underscore _
 - Replace **L01** with the correct lab number each week
 - All projects from this lab should be created inside this folder

CREATING YOUR PROJECTS

Every lab exercise tells you which name you should use. Use the provided prog1 cmake project zip files under 00_INSTALL. Instructions on how to use can also be found in this location.

EX01: HANDS-ON DEMO: VARIABLES

PROJECT SETUP

- Create a new **PROG1 CONSOLE PROJECT** in this lab's folder, using the prog1 console project zip file.
 - Call this project **EX01_Variables**
- Your name, first name and group should be mentioned as comment at the top of each **cpp** file.

```
// Name, Firstname - 1DAEXX
```

- Replace XX with your group number

Naming Conventions & Core Guidelines

Check the new [Naming Conventions and Core Guidelines](#) that apply for this week's lab.

HANDS-ON DEMO

The following steps are shown and explained by the teacher in a demo, take notes if needed.

- How to **create** the project.
- How to **build** a project.
- How to **declare** local variables and **initialize** them.
- How to **write to the console** using the insert operator `<<` and `std::cout`.
- How to **read from the console** using the extract operator `>>` and `std::cin`.
- How to indicate that lines must be considered as **comment** by the build process.
- How to define/delete a **breakpoint** and why it is useful.
- How to **solve errors** reported by the build process.
- How to **upload** your solutions.

EXERCISE: FINISH THE DEMO CODE

Now **continue in your Ex01_Variables project from the demo** as instructed below. Check which steps were already covered in the demo and add / change code where needed.

Note: you will have to understand the [difference between declaring and initializing a variable](#) as handled in theory; take your notes with you if necessary.

DEFINING/DECLARING VARIABLES WITHOUT INITIALIZATION

In the **main** function, define following variables **without initializing** them:

- integral types: int, char, bool
- floating types: float, double
- Mind using the correct naming convention for variables.
- Write code to send their value to the console.
- Compile the cpp file.
- You get errors: **In C++ you always need to initialize variables, giving them an initial value.**
 - Comment out (do not remove) the lines that cause issues.

DEFINING/DECLARING VARIABLES WITH DEFAULT INITIALIZATION

Change the code:

- Give the variables a value during definition using the default initialization (see theory)
- Look at their values using **a break point (!)** and stepping through the code
- Look at their values in the **watch window** of visual studio
- Show their values on the **console** surrounded by parenthesis.

DEFINING/DECLARING VARIABLES WITH LITERAL CONSTANTS INITIALIZATION

- Now give them a value during definition again using the {} initialization but passing a **literal** constant value.
 - More info is available on [Literals](#)
- Show the content of the variables on the console again, surrounded by parenthesis.

ASSIGNING A LITERAL CONSTANT USING THE ASSIGNMENT OPERATOR

You can change the content of a variable **after** initializing it, by using the **assignment operator**.

- Give the variables another value using the assignment operator (=)
- Again, show their content on the console.

ASSIGNING A USER'S INPUT VALUE USING THE "GET FROM" (>) OPERATOR

- Ask the user to enter a value for each variable type.
- Show the resulting value on the console.
- For example, for the char type:

```
Please enter a character value:
```

→ example result after the user entered R:

```
Please enter a character value: R  
-> The character that was entered by the user is R.
```

EX02: EXPRESSIONS

PROJECT SETUP

- Create a new **PROG1 CONSOLE PROJECT** using the template zip file again (EX00_ConsoleProjectName.zip, installation **step 3/4**)
 - Call this project **EX02_Expressions**
- Your name, first name and group should be mentioned as comment at the top of each **cpp** file.

```
// Name, Firstname - 1DAEXX
```

- Replace XX with your group number

*Before you start, make sure you are familiar with the terminology used in theory:
Expressions, literals, operators and operands, assignment operator =*

- **The main goal of this part is to experiment with various expressions** in your code! Try, change, try again, use breakpoints and console inputs and outputs to test!

BINARY OPERATORS +, -

Binary operators require 2 operands, hence the word “binary”. These operands can be literals, variables or the result of another expression.

- Build expressions using these operators and send the result to the console using the send to (<<) operator.
- Experiment with combining several **operand** types:
 - integer variables and literalsfor example:

```
int number{ 4 }; // integer variable

//integer variable (number) + literal (9):
std::cout << "The result of number + 9 = ";
//TODO: add the correct expression in above code to show result
```

```
The result of number + 9 = 13
```

- float variables and literals
- char variables and literals
- mixture of char and int variables and literals

BINARY OPERATORS *, /

Experiment with various operand types in code:

- integer variables and literals
- float variables and literals
- mixture of int and float variables and literals
- check what happens when you divide an integer by a smaller integer.

COMPOUND EXPRESSIONS

- Make some exercises on **compound expressions** (= combination of expressions).
- Make sure you experiment with the order in which they are carried out!
 - You can read more about this subject on [Operator precedence and associativity](#)

COMPOUND ASSIGNMENT AND UNARY OPERATORS

- Experiment with **compound assignment operators** (see theory).
 - Which operators can use this principle?
- Experiment with **unary operators**.
 - Which operators can use this principle?
 - What happens if you use then in compound expressions?
 - Try placing the unary operators before + after the var in an expression; what's the difference?

EX03: MAKING FRIENDS

```
The person sitting to my left is called Erica
Erica is from America
The age of Erica from America is 17
The person sitting to my right is called Ali
Ali is from Bali
The age of Ali from Bali is 23
The average age of Erica and Ali is 20
```

```
The person sitting to my left is called Timothee
Timothee is from France
The age of Timothee from France is 28
The person sitting to my right is called Zendaya
Ali is from the United States
The age of Zendaya from the United States is 27
The average age of Timothee and Zendaya is 27.5
```

Time to get to know your classmates! Create a **Prog1 console project** (from zip file) called **EX03_MakingFriends** and try to accomplish the above. Make sure to ask the actual name, age and country of your neighbor students.

- The **yellow text is the input** that the user enters in the console; this can change. Try various values!
 - › **Note:** the type for a text variable is `std::string`; we need the `std::` before it!
 - › *You do **not** need to make the text yellow; this is just to indicate here what is user input.*
- Study the examples above: each line asks for an input, then uses the input value(s) in the next line(s).
 - › Create a **variable** for every input *and* for every calculated value!
 - › Alternate between writing with `std::cout`, and reading with `std::cin`. Input and output statements must remain separate and **cannot** be combined into a single statement.
- Once it works, **try to enter a full name**, for example:

```
The person sitting to my left is called Zendaya Maree Stoermer Coleman
```

- › What happens?
- › *A whitespace indicates the end of an input by default.*
- › *This means the above example has 4 input values.*

PART 2: FAVORITE COLOR

```
The person sitting to my left is called Erica
Erica is from America
The age of Erica from America is 17
The person sitting to my right is called Ali
Ali is from Bali
The age of Ali from Bali is 23
The average age of Erica and Ali is 20

Ali loves the color hot_pink
The RGB value of hot_pink is: 255 0 100
The normalized RGB value of hot_pink is (1, 0, 0.392157)
```

We will continue this exercise by also asking the favorite color:

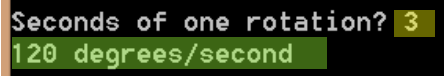
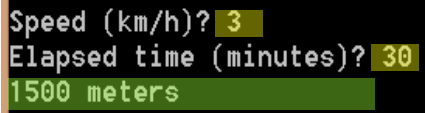
- ➔ Color pickers give Red Green Blue values in a range between 0 and 255.
- ➔ Game engines, however, (like the one you will use in Prog1) tend to work RGB values in range 0 and 255

- › Ask for the favorite color of one of your neighbors
- › Ask them to provide the exact RGB value [0-255] of this color
- › Calculate and output the RGB values in range [0-1]

EX04: APPLICATIONS

- Create a new **PROG1 CONSOLE PROJECT** using windows desktop wizard in this lab's folder (see [EX0](#))
 - Call this project **EX04_Applications**
 - Your name, first name, group should be mentioned as comment at the top of each **cpp** file.
- **Requirements:**
 - **Create variables** for input values **and** to store the result of any calculations.
 - *Yes, even if for now you only need the result to show up in the console.*
 - Do not use **magic numbers**; replace them by meaningful variables
 - `frameTime = 1.0f / 60.0f;` // bad: what is this 60 value?
 - `float framesPerSecond{ 60.0f };`
 - `frameTime = 1.0f / framesPerSecond;` // good: readable, reusable

The table below lists some practical exercises. Yellow is example input, green is example output. Implement them:

Requested	Example screenshot
Your level design doc measures distances in kilometers ("the player walks 2.5 km to the next checkpoint"). Your engine works in meters and centimeters. Write a program that takes a designer's distance and converts it for the engine.	
Your physics engine rotates objects using radians internally, but nobody on the design team thinks in radians. Write a program that lets someone type an angle the way the physics engine stores it, and see it the way a human would read it.	
Same problem, other direction: now a designer wants to type an angle the human way and see what the physics engine would store.	
A rotating platform in your level completes one full spin every few seconds. A level designer will tell you how many. Your physics engine needs the angular velocity of that platform in degrees per second. Write the program that gives it to them.	
An enemy on patrol walks at a constant speed (km/h). You need to know how far it will have travelled after a given number of minutes, to decide if it reaches its patrol waypoint in time.	
A physics prop (a barrel, say) starts falling in free fall accelerating downwards at a rate of 9.8 m/s². The designer enters how many seconds this prop is already falling, your physics code then calculates its current velocity.	

An enemy's detection range is a circle with a given radius. Design wants two numbers out of it: how far a player has to run to skirt the entire edge of it, and how much ground area it actually covers.

```
Radius of circle? 10
Circumference: 62.8319
Area: 314.159
```

EX05: AWAITS HUMAN REVIEW

Unzip the **given cmake project** with the same name as the title above.

You have been given a **piece of AI-generated C++ code**. Your task is to give it a code review and identify mistakes, bad practices, guideline violations, and logic errors.

Your goal

- Use the [Core guidelines & naming conventions](#) section at the start of this document as reference.
- For every guideline violation, **mention the corresponding guideline code** (e.g. ES.23, NL.5).
- **Write your findings as comments** above or next to the relevant line(s) of code.
- Always **explain in short** why the code is problematic, not just what is wrong.
- For each issue you find, comment out original code and **write the corrected version underneath it**.

IMPORTANT

IF YOU ARE PLANNING TO ASK AN AI TO REVIEW THE AI-GENERATED CODE FOR YOU, PLEASE SKIP THIS ASSIGNMENT ENTIRELY. YOU'LL SAVE:

- *YOUR TIME,*
- *OUR TIME,*
- *A FEW SERVERS,*
- *AND POSSIBLY AN ENTIRE FOREST WORTH OF ELECTRICITY.*

More importantly, you'll miss the actual learning goal: developing the ability of critical thinking. In a few weeks, you won't be graded on your ability to ask an AI for answers. You'll be graded on your ability to review code yourself. This will be done in a closed-book environment, without AI, notes, or online resources.

OPTIONAL: MAKING FRIENDS CONTINUED – ANSI CODES

```
The person sitting to my left is called Erica
Erica is from America
The age of Erica from America is 17
The person sitting to my right is called Ali
Ali is from Bali
The age of Ali from Bali is 23
The average age of Erica and Ali is 20
```

NOTE: This section is **optional** and aimed at curious students who enjoy digging a little deeper. It covers additional topics beyond the course material and may require some independent research. It is **not** required knowledge and will **not** be examined.

- Look for information about **ANSI codes**
(if you're going to use AI instead of old-school search engines, make sure you only ask what ANSI escape codes are and how they work, **not** which exact codes to use or how to solve this exercise)
- Create variables for numbers of colors, color code strings, etc.
- Try to accomplish a similar result to the screenshot. Play around more with ANSI codes if you please!

EX06: SDLANDOPENGL (SECOND LAB BLOCK, HANDS-ON DEMO)

This project is an introduction to SDL and OpenGL. Two libraries that help us with keyboard and mouse input and drawing operations.

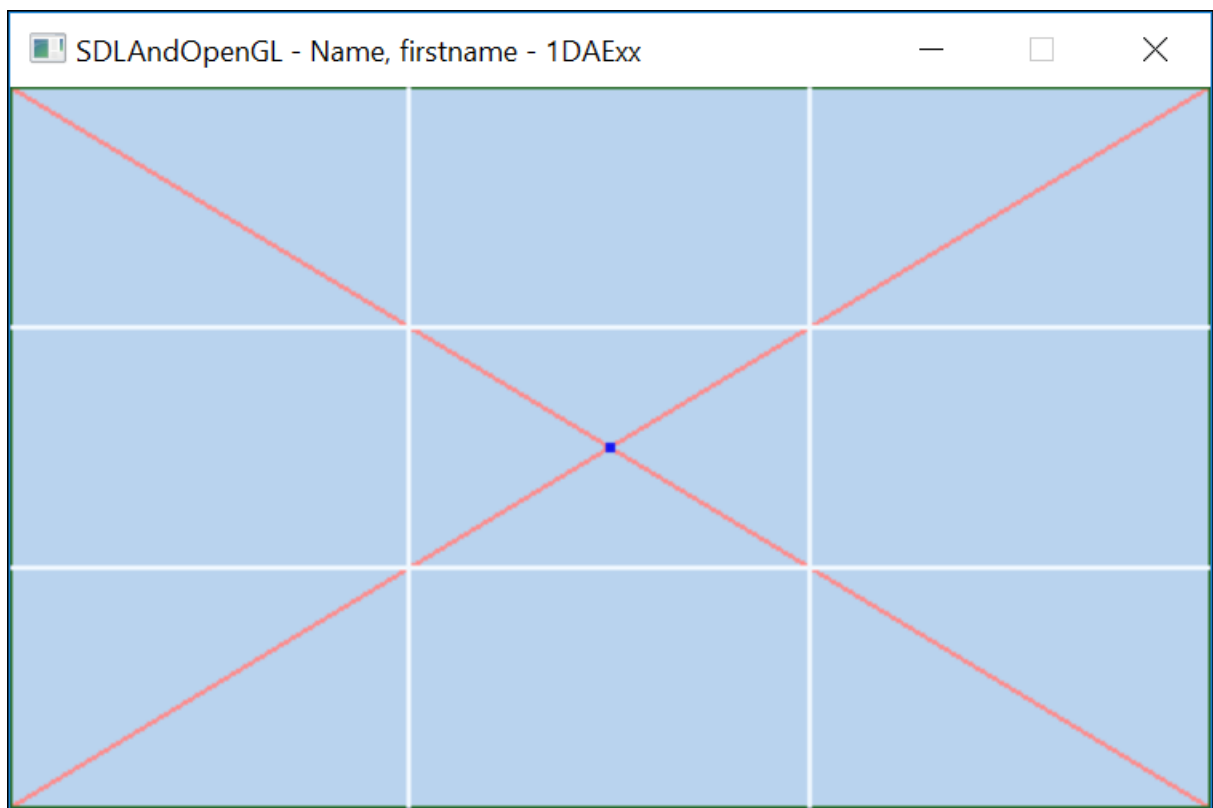
First look at the teacher demo that demonstrates how to do drawing operations using our framework.

Create a new **Prog1 Game Project** with name `Ex06_SdlAndOpenGL` in this week's folder. Check the instructions under 00_INSTALL on Leho if you do not know how to do this.

Try to find where you can change the window title and size. Change both.

Demo: The teacher will you show how to e.g. use colors and draw a line and a rectangle.

Comment the demo drawing code and complete the project on your own with the necessary drawing instructions in `engine/utils.h` to get following result.



Notice the:

- Grayish background
- Green border
- 2 red diagonals
- 2 white vertical and 2 white horizontal lines that split the screen in 9 areas of the same size
- A blue point with size 4 in the center.

Use the following commands:

- **SetColor** to change the color to be used for the following drawings
- **DrawLine** to draw a ... line!
- **DrawRect** to draw a rectangle

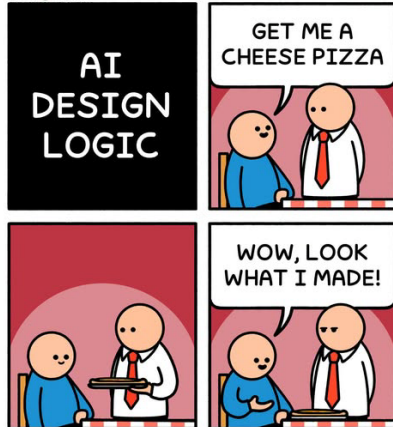
EX07: CODING CRAFTSMANSHIP CHALLENGE

DO NOT SKIP THIS EXERCISE; EXPERIMENTING WITH CODE IN A SELF-MADE PROJECT IS BY FAR THE BEST WAY TO MASTER PROGRAMMING!

INSTRUCTIONS

Create either a **Prog1 Game Project** called **CodingCraft01**.

TWOONS



Create a **small original application** that demonstrates the concepts we've covered so far.

- ✓ Create **your own** original project.
- ✓ **Only** use **concepts that have been covered** in labs already.
- ✓ Write code you can explain yourself.
- ✗ **No** AI-generated code.
- ✗ **No** copied solutions from the internet.
- ✗ **No** copying another student's work.

For example, this week: draw your favorite logo or character using basic shapes or find inspiration by searching for daily minimal in Google images.

WALL OF FAME

Every week, some projects will earn a place on the lab's Wall of Fame.

A project does not need to be complex to deserve a spot. Original ideas, clever concepts, fun themes, polished outputs, and well-written code are just as valuable. Projects are shown **anonymously** so you can choose whether you reveal yourself as the creator or not.

The **goal** is to **learn, experiment, and become a better programmer**. The Wall of Fame only works if every project genuinely reflects your own work. **Let your imagination run wild and work on a programming project YOU love!**

SUBMISSION INSTRUCTIONS

You must submit **two files**:

- A **zip file** containing your **complete lab folder** of this week with all exercises.
- **CodingCraft01.zip** containing only your Coding Craftsmanship Challenge project.

BEFORE ZIPPING

For every project:

- **Close** Visual Studio.
- Delete the **out** folder.
- Delete the hidden **.vs** folder.

SUBMIT

- Upload **both files** with the correct assignment before the start of next week's first lab.
- Make sure to complete the **quiz**.
- *These steps are required to unlock the next lab assignment!*

⚠ Your submission may be discussed during a code review. Make sure you understand and can explain your own code.

REFERENCES

STRUCTURE OF A PROGRAM

http://www.cplusplus.com/doc/tutorial/program_structure/

COUT, CIN, ENDL, STD, USING

A first look at cout, cin, endl, the std namespace, and using statements.

<http://www.learncpp.com/cpp-tutorial/1-3a-a-first-look-at-cout-cin-endl-namespaces-and-using-statements/>

LITERALS

<http://www.learncpp.com/cpp-tutorial/28-literals/>

OPERATOR PRECEDENCE AND ASSOCIATIVITY

<http://www.learncpp.com/cpp-tutorial/31-precedence-and-associativity/>